OpenSDN vRouter Agent tutorial

By Matvey Kraposhin

kraposhin.online

April 18, 2026

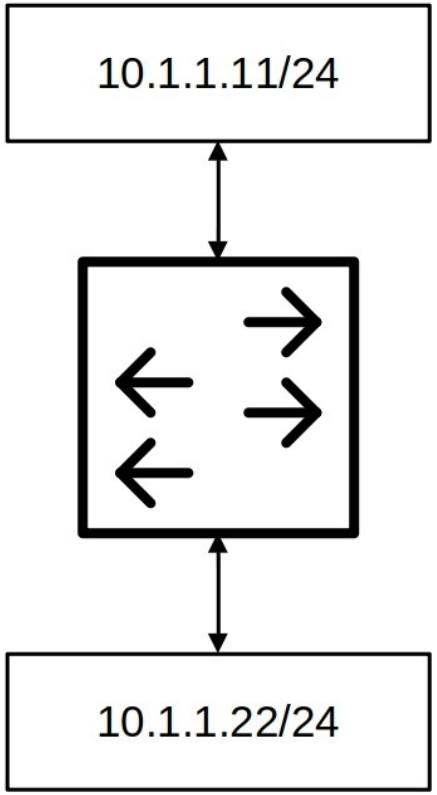
Objectives of the lab

- Demonstration of interaction between OpenSDN Controller and OpenSDN vRouter Agent modules.
- Upon completion of the lab, a student will learn:
  - how the IF-MAP graph defines virtual network environments;
  - how an IF-MAP graph (a configurative state) is translated into FIB;
  - how vRouter Agent is managed by vRouter Controller messages.

The lab’s prerequisites

- The previous lab contains sections with instructions on how to prepare the initial virtual environment (sections A, B and C) for this lab.
- Hence, it is assumed that a student completed the previous lab (or sections A, B and C, at least): https://github.com/mkraposhin/opensdn-forwarder-basic-tutorial

The considered problem



- A communication between two endpoints is considered.
- These endpoints belong to 2 containers located in a host OS.
- Their connectivity is implemented using OpenSDN vRouter Forwarder.
- The package-based forwarding regime is used.

The lab steps

1) Study materials of the lab: the presentation and the repository.
2) Prepare the environment for the lab.
3) Prepare the input data (IF-MAP graph).
4) Running the configuration.
5) RIB/FIB checks, connectivity tests.

The lab’s materials

- All materials are located in the repository: https://github.com/mkraposhin/opensdn-agent-basic-tutorial
- A set of instructions (an algorithm) to complete the lab is can be found in this presentation, or in a shorter form, here: https://github.com/mkraposhin/opensdn-agent-basic-tutorial/blob/main/steps.md

The lab’s algorithm overview

- There are 12 steps to complete in the lab:
  - step 0 (the initial state, next slide);
  - steps 1 and 2: input data preparation;
  - steps 3 to 7 describe how to run vRouter Forwarder and vRouter Agent manually;
  - steps 8 to 10 explain how to load virtual network configuration into vRouter Agent;
  - steps 11 and 12 contain actions to check connectivity between containers.

All steps will be completed under the lecturer’s supervision given that listeners have the working initial state.

The initial state

- The state corresponds to the state obtained after the completion of sections A, B and C of vRouter Forwarder basic tutorial:
  - vRouter Forwarder kernel module has been loaded into the memory;
  - opensdn-tools has been started;
  - containers cont1 and cont2 has been started;
  - virtual interfaces linking **cont1** and **cont2** with the host OS have been created and attached to the vRouter Forwarder.

The initial state: “**ip a**” output

- Veth1 and veth2 interfaces must be created (see basic vRouter Forwader tutorial, https://github.com/mkraposhin/opensdn-forwarder-basic-tutorial

```
10: pkt1: <> mtu 65535 qdisc noop state DOWN group default qlen 1000
    link/void 72:bd:7c:bb:d0:b5 brd 00:00:00:00:00:00
11: pkt3: <> mtu 65535 qdisc noop state DOWN group default qlen 1000
    link/void da:86:1b:76:d6:63 brd 00:00:00:00:00:00
12: pkt2: <> mtu 65535 qdisc noop state DOWN group default qlen 1000
    link/void ba:82:cc:80:75:7f brd 00:00:00:00:00:00
14: veth1@if13: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 52:df:90:a2:68:d4 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::50df:90ff:fea2:68d4/64 scope link
       valid_lft forever preferred_lft forever
16: veth2@if15: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether ea:47:ff:70:44:a4 brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet6 fe80::e847:ffff:fe70:44a4/64 scope link
       valid_lft forever preferred_lft forever
opensdn26@opensdn26-VirtualBox:~$
```

The initial state: **vif** output

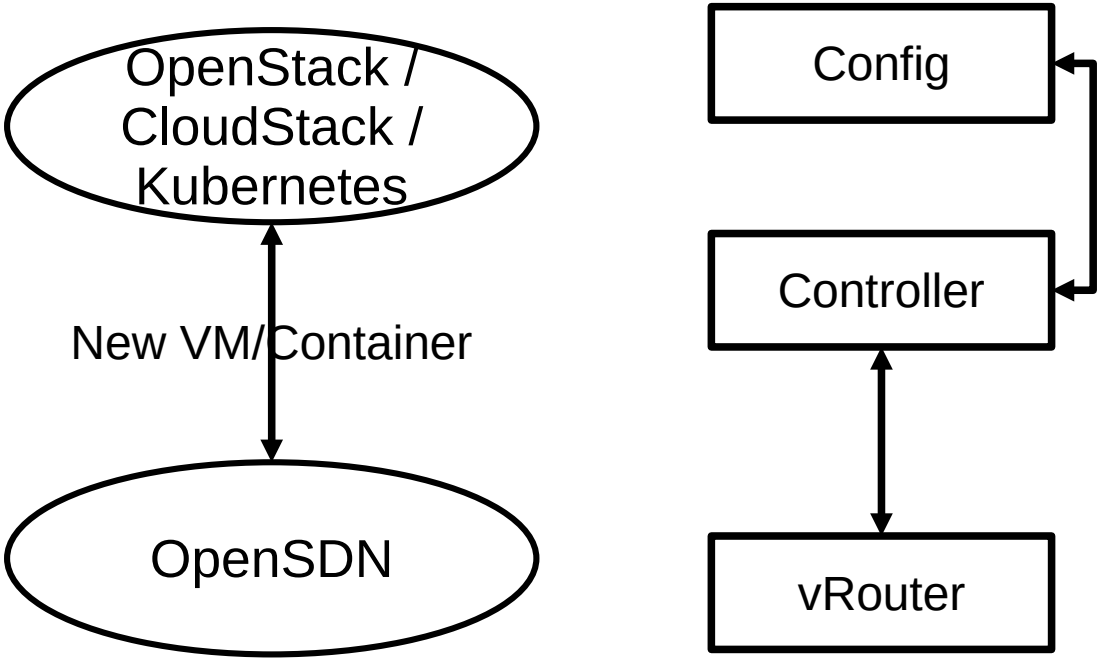
- **veth1** and **veth2** interfaces must be connected to the vRouter Forwarder

```
@opensdn26-VirtualBox:/
opensdn26@opensdn26-Vi...   @opensdn26-VirtualBox:/   root@4d591cf0973e: /   root@481a938c0a31: /   opens
opensdn-tools)[root@opensdn26-VirtualBox /]$
opensdn-tools)[root@opensdn26-VirtualBox /]$ vif --list
router Interface Table

lags: P=Policy, X=Cross Connect, S=Service Chain, Mr=Receive Mirror
      Mt=Transmit Mirror, Tc=Transmit Checksum Offload, L3=Layer 3, L2=Layer 2
      D=DHCP, Vp=Vhost Physical, Pr=Promiscuous, Vnt=Native Vlan Tagged
      Mnp=No MAC Proxy, Dpdk=DPDK PMD Interface, Rfl=Receive Filtering Offload, Mon=Interface is Monitored
      Uuf=Unknown Unicast Flood, Vof=VLAN insert/strip offload, Df=Drop New Flows, L=MAC Learning Enabled
      Proxy=MAC Requests Proxied Always, Er=Etree Root, Mn=Mirror without Vlan Tag, HbsL=HBS Left Intf
      HbsR=HBS Right Intf, Ig=Igmp Trap Enabled, Ml=MAC-IP Learning Enabled

if0/14     OS: veth1
           Type:Virtual HWaddr:00:00:5e:00:01:00 IPaddr:0.0.0.0
           Vrf:0 Mcast Vrf:65535 Flags:L3L2 QOS:0 Ref:1
           RX packets:16  bytes:2091 errors:0
           TX packets:0  bytes:0 errors:0
           Drops:16

if0/16     OS: veth2
           Type:Virtual HWaddr:00:00:5e:00:01:00 IPaddr:0.0.0.0
           Vrf:0 Mcast Vrf:65535 Flags:L3L2 QOS:0 Ref:1
           RX packets:18  bytes:2267 errors:0
           TX packets:0  bytes:0 errors:0
           Drops:18
```

vRouter Agent external connections

Two data sessions are needed:

1) a request about attachment of a VM and a VMI;
2) a configurative state update with the description of changes in a virtual network environment.



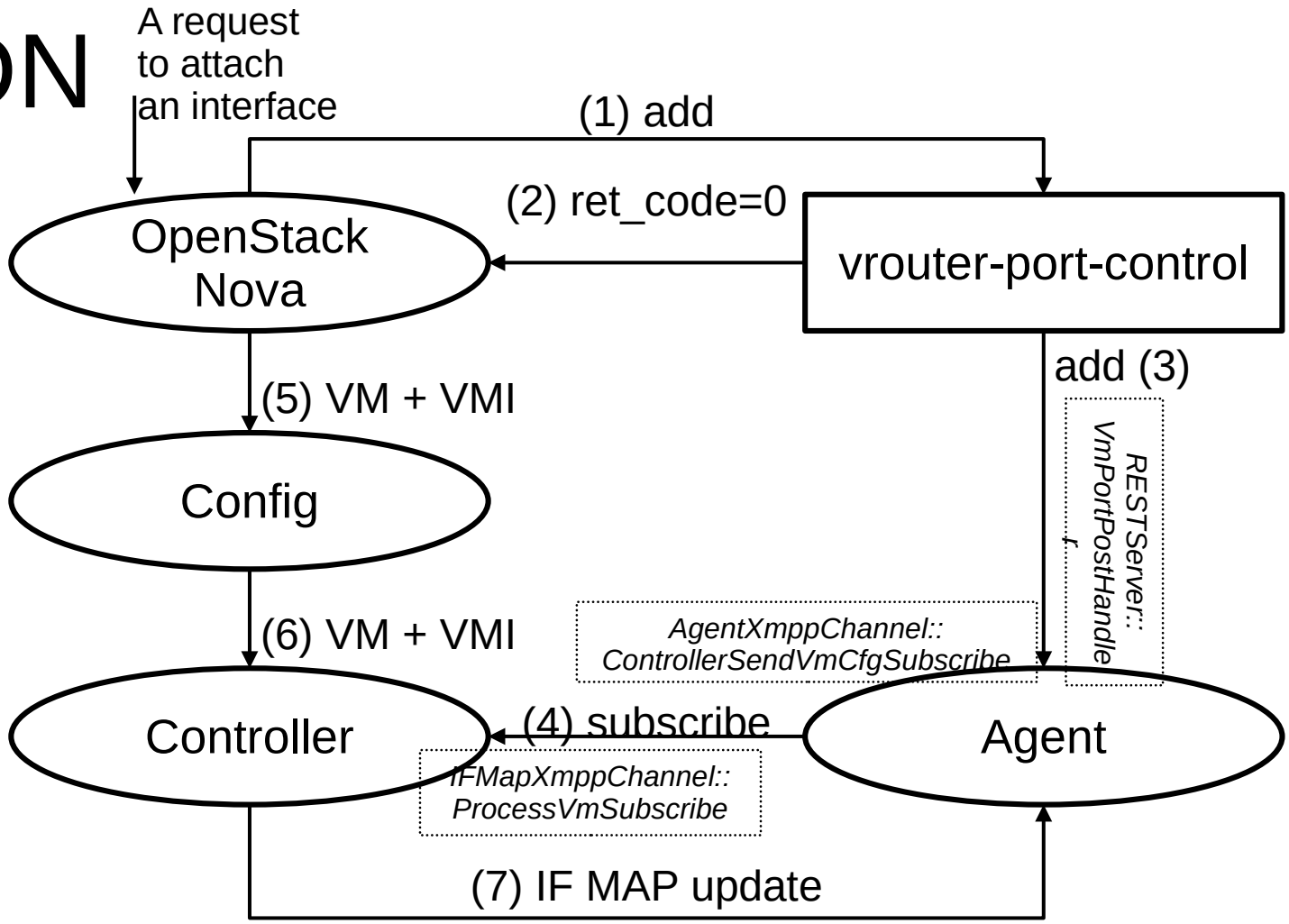
OpenSDN data model

- OpenSDN vRouter Agent uses a configurative state (governed by the low-level data model) stored as an IF-MAP graph to infer an operative state.
- Then the operative state is transferred into the FIB to feed into the vRouter Forwarder.

This slide corresponds to step 1 from steps.md

How a virtual interface is connected to OpenSDN

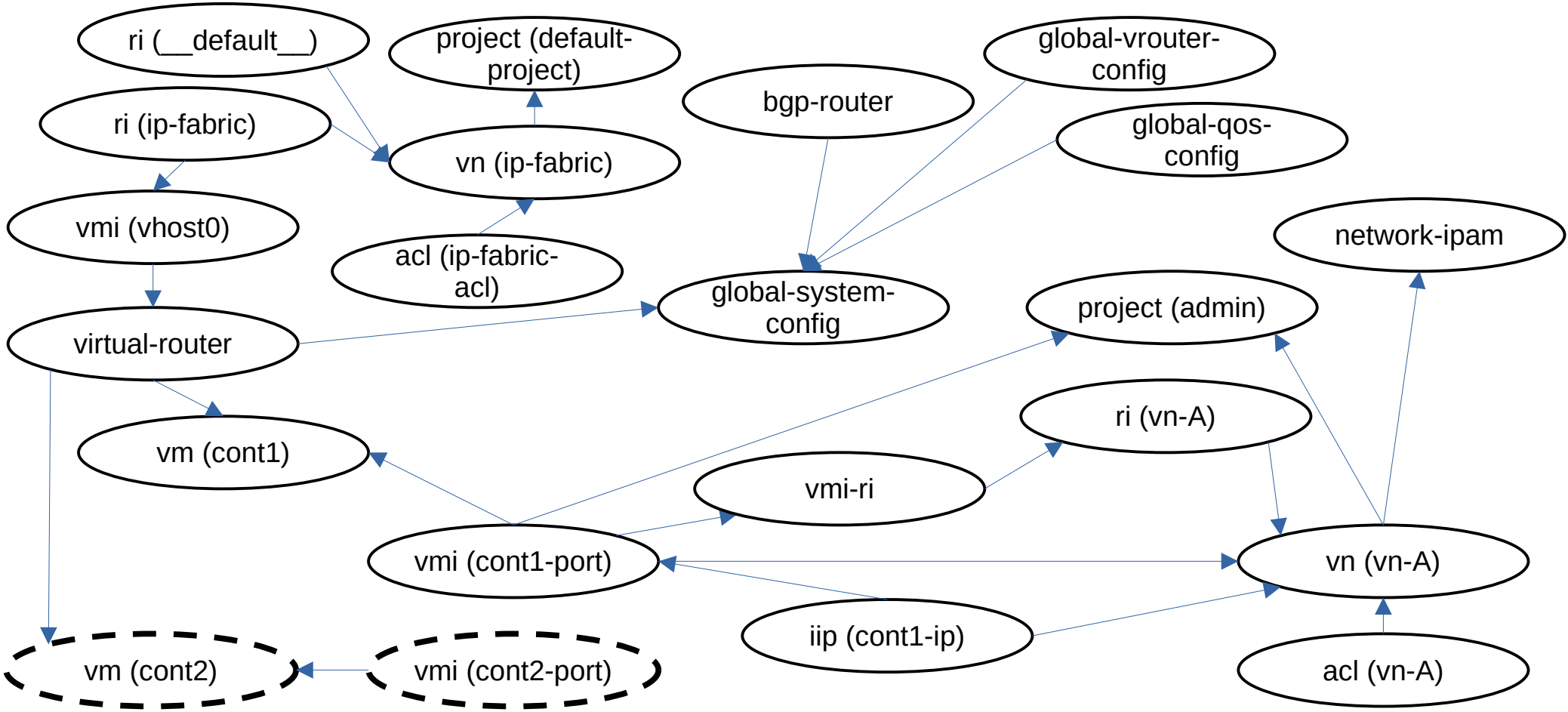
- Actions (5) and (4) can be executed in parallel.
- **In this lab we reproduce (7) and (3).**



The lab’s configurative state

- The configuration (configurative state) contains:
  - objects to store global settings (global-system-config, global-vrouter-config, etc);
  - a default network and VRF to store underlay routes (virtual-network ip-fabric and neighbours);
  - a project (admin) and a virtual network (vn-A) to connect the containers;
  - port (virtual-machine-interface) and virtual machine (virtual-machine) objects to define connectivity between containers.

The configuration implementation



The abbreviations used for the implementation

- vmi – virtual-machine-interface
- vm – virtual-machine
- vn – virtual-network
- ri – routing-instance (a VRF table)
- acl – access-control-list
- iip – instance-ip
- vmi-ri - virtual-machine-interface-routing-instance

How the lab’s configurative state is stored

- Download the repository with the lab into the home folder (see step 2 from steps.md).
- The configurative state is stored as 5 XML files:
  - global-config.xml stores node “global-system-config” and its direct neighbours;
  - default-project.xml stores node “project (default-project)” and its main neighbours;
  - admin-project.xml stores “project (admin)” node and its main neighbours;
  - cont1.xml and cont2.xml store information about nodes “vm (cont1)” (and “vm (cont2)”) and their connections to other nodes that reflect attachment of the containers via the respective interfaces to virtual network vn-A.

This slide corresponds to step 2 from steps.md

global-config.xml, 1

- The name of the agent node, the recipient of messages (where the agent process is running):
  - “to” field (opensdn-VirtualBox in the example), can be obtained using “hostname” command;
  - must be adjusted in <iq>, <node type="virtual-router">, <node type="virtual-machine-interface">, <link>;
- virtual-router-ip-address: IP address of the hypervisor (enp0s3/vhost0 IP);
- ip-prefix of the hypervisor: (usually equal to virtual-router-ip-address with /24).

global-config.xml

```
<iq type="set" from="network-control@contrailsystems.com" to="default-global-system-config:opensdn-VirtualBox/config">
    <config>
```

```
<name>default-global-system-config:opensdn-VirtualBox</name>
<virtual-router-dpdk-enabled>false</virtual-router-dpdk-enabled>
<virtual-router-ip-address>10.0.2.15</virtual-router-ip-address>
```

```
<ip-fabric-subnets>
    <subnet>
        <ip-prefix>10.0.2.0</ip-prefix>
        <ip-prefix-len>24</ip-prefix-len>
    </subnet>
</ip-fabric-subnets>
```

global-config.xml, 2

- mac-address the hypervisor: (it is equal to the MAC of vhost0/enp0s3)
- identifier and address of the controller (equal to ip-prefix in the lab)
- ip-fabric-service-ip

```
<virtual-machine-interface-mac-addresses>
  <mac-address>08:00:27:20:f9:ed</mac-address>
</virtual-machine-interface-mac-addresses>
```

```
<autonomous-system>64512</autonomous-system>
<identifier>10.0.2.15</identifier>
<address>10.0.2.15</address>
```

```
<ip-fabric-DNS-service-name></ip-fabric-DNS-service-name>
<ip-fabric-service-port>8775</ip-fabric-service-port>
<ip-fabric-service-ip>10.0.2.15</ip-fabric-service-ip>
```

default-project.xml

- The name of the agent node, the recipient of messages (where the agent process is running):
  - “to” field (opensdn-VirtualBox in the example);
  - other occurrences, for example, in <node type="virtual-machine-interface-routing-instance">

```
<iq type="set" from="network-control@contrailsystems.com" to="default-global-system-config:opensdn-VirtualBox/config">
    <config>
        <update>
            <node type="virtual-machine-interface-routing-instance">
                <name>attr(default-domain:default-project:ip-fabric:ip-fabric,default-global-system-config:opensdn-VirtualBox:vhost0)</name>
                <value>
```

admin-project.xml

- The name of the agent node, the recipient of messages (where the agent process is running):
  - “to” field (opensdn-VirtualBox in the example).

global-config.xml admin-project.xml default-project.xml

```
1 <iq type="set" from="network-control@contrailsystems.com" to="default-global-system-config:opensdn-VirtualBox/config">
2     <config>
3         <update>
4             <node type="project">
5                 <name>default-domain:admin</name>
```

cont1.xml

- The name of the agent node, the recipient of messages (where the agent process is running):
  - “to” field (opensdn-VirtualBox in the example);
  - other occurrences, for example, in <key-value-pair><value>, etc.
- MAC address of **veth1** interface (inside **cont1** container), e.g. <mac-address>76:84:d4:e1:04:75</mac-address>

global-config.xml | admin-project.xml | default-project.xml | cont1.xml

```
<iq type="set" from="network-control@contrailsystems.com" to="default-global-system-config:opensdn-VirtualBox/config">
    <config>
        <key>host_id</key>
        <value>opensdn-VirtualBox</value>
    </key-value-pair>
<virtual-machine-interface-mac-addresses>
    <mac-address>76:84:d4:e1:04:75</mac-addr
</virtual-machine-interface-mac-addresses>
<link>
    <node type="virtual-router">
        <name>default-global-system-config:opensdn-VirtualBox</name>
    </node>
```

cont2.xml

- For the **cont2** container, changes are similar to cont1.xml, except MAC address of **veth2** interface, its value must be take from **cont2** container.
- The value in the example is: <mac-address>ca:20:76:c1:2c:89</mac-address>

run-port-control-cont1 and run-port-control-cont2

- MAC addresses in the scripts must be set according to the values provided for cont1.xml and cont2.xml
- --mac=7a:ab:8d:0b:4d:06 \

```
    --ip_address=10.10.10.11 \
    --ipv6_address= \
    --vm_name=cont1 \
    --mac=76:84:d4:e1:04:75 \
    --tap_name=veth1 \
    --port_type=NovaVMPort \
```

```
    --ip_address=10.10.10.22 \
    --ipv6_address= \
    --vm_name=cont2 \
    --mac=ca:20:76:c1:2c:89 \
    --tap_name=veth2 \
    --port_type=NovaVMPort \
```

Objects’ UUIDs

- All objects have unique UUID, their value for the configuration are stored in the file settings/UUIDs for reference.
- No changes are needed.

Installing vRouter Agent

- This step corresponds to step 3 from steps.md.
- It involves modification of the vRouter Agent container:
  - sudo docker pull opensdn/opensdn-vrouter-agent
  - sudo docker create --privileged --pid host --net host --name agent opensdn/opensdn-vrouter-agent:latest
  - sudo docker cp tut-agent/scripts/entrypoint.sh agent:/
  - sudo docker start agent

Step 3 screenshots

- Upon completion of step 3, the terminal should look like the provided screenshots.

```
opensdn26@opensdn26-VirtualBox:~$ sudo docker pull opensdn/opensdn-vrouter-agent
[sudo] password for opensdn26:
Using default tag: latest
latest: Pulling from opensdn/opensdn-vrouter-agent
02bfbe80c318: Pull complete
d0861e55c9c8: Pull complete
6ad6f0eaac3c: Pull complete
575573787eea: Pull complete
9e7bce06149b: Pull complete
2987f05d42cb: Pull complete
8c857bb1f77f: Pull complete
dc233a14bb1b: Pull complete
Digest: sha256:2d40108f6124115c2e7d3cdd19ed9322f42ee824acf3307366aa25e4a1a9832a
Status: Downloaded newer image for opensdn/opensdn-vrouter-agent:latest
docker.io/opensdn/opensdn-vrouter-agent:latest
```

```
opensdn26@opensdn26-VirtualBox:~$ sudo docker create --privileged --pid host --net host --name agent opensdn/opensdn-vrouter-agent:latest
05cbb4a45a387f68c8c2ced591943721d497d6a5daa53c458bc7737d72e41584
opensdn26@opensdn26-VirtualBox:~$ sudo docker cp tut-agent/scripts/entrypoint.sh agent:/
sudo docker start agent
Successfully copied 2.05kB to agent:/
agent
opensdn26@opensdn26-VirtualBox:~$
```

Getting vrouter-port-control

- In the home dir ($HOME)
- wget https://raw.githubusercontent.com/OpenSDN-io/tf-controller/refs/heads/master/src/vnsw/agent/port_ipc/vrouter-port-control
- This is step 4 from steps.md

controller-reply.xml

- When an agent sends “VR subsrcibe” message to a controller, the latter must answer with a kind of “ACK”, “to” and “from” fields must be corrected according to settings in other XML files.
- The agent sends:
  - <stream:stream from="opensdn-VirtualBox" to="network-control@contrailsystems.com" version="1.0" xml:lang="en" xmlns="" xmlns:stream="http://etherx.jabber.org/streams" >
- The controller must reply (this is provided by a user in the lab):
  - <stream:stream from="network-control@contrailsystems.com" to="opensdn-VirtualBox" id="++123" version="1.0" xml:lang="en" xmlns="" xmlns:stream="http://etherx.jabber.org/streams" >
- This is step 5 from steps.md

Running the configuration

- This stage corresponds to steps 6 – 10.
- Detailed explanations are on the next slides.

Running controller

- This is step 6 from steps.md
- The controller is imitated using “nc” program:
  - sudo nc -4kl -s 10.0.2.15 -p 5269
- Whenever the controller must send a message to the agent, this is imitated by submitting the prepared messages (step 2 from steps.md) manually into the nc.

Running vRouter Agent

- This is step 7 from steps.md:
  - sudo docker exec -ti agent bash
  - # (inside the agent's container)
  - source /actions.sh
  - prepare_agent # (vhost0)
  - create_agent_config
  - /usr/bin/contrail-vrouter-agent

Step 7 screenshots, 1

The terminal should look like the screenshots on this and next slide after completing step 7.

```
INFO: sriov parameters are not provided
INFO: wait vhost0 to be initilaized... 0/60
INFO: vhost0 is ready.
INFO: Start prepare_agent_config_vars
INFO: vhost0 cidr 10.0.2.15/24
INFO: Physical interface: enp0s3, mac=08:00:27
INFO: vrouter gateway: 10.0.2.2
(vrouter-agent)[root@opensdn26-VirtualBox /]$
```

```
opensdn26@opensdn26-VirtualBox:~$ sudo docker exec -ti agent bash
(vrouter-agent)[root@opensdn26-VirtualBox /]$ source /actions.sh
(vrouter-agent)[root@opensdn26-VirtualBox /]$ prepare_agent # (vhost0)
INFO: ================== Mon Mar  9 07:01:13 PM UTC 2026 ==================
INFO: Start set_traps
INFO: Start vhost0_init
Device "vhost0" does not exist.
cat: /sys/class/net/vhost0/address: No such file or directory
INFO: creating vhost0 for nic mode: nic: enp0s3, mac=08:00:27:13:0a:e3
INFO: /sys/class/net/pkt1/queues/rx-0/rps_cpus
1
INFO: /sys/class/net/pkt1/queues/rx-1/rps_cpus
2
INFO: /sys/class/net/pkt2/queues/rx-0/rps_cpus
1
INFO: /sys/class/net/pkt2/queues/rx-1/rps_cpus
2
INFO: /sys/class/net/pkt3/queues/rx-0/rps_cpus
1
INFO: /sys/class/net/pkt3/queues/rx-1/rps_cpus
2
INFO: there is no ifcfg-enp0s3 and ifcfg-vhost0, so initialize vhost0 manually
INFO: Changing physical interface to vhost in ip table
Error: ipv6: address already assigned.
ERROR: failed to add address fe80::a00:27ff:fe13:ae3/64 to vhost0.
INFO: set mtu
INFO: ip route replace 10.0.2.0/24 proto kernel scope link src 10.0.2.15 metric 100  dev vhost0
INFO: ip route replace default via 10.0.2.2 proto dhcp src 10.0.2.15 metric 100  dev vhost0
modinfo: ERROR: Module alias vrouter not found.
INFO: versions: agent=v0+nightly, loaded_vrouter=v0+nightly, available_vrouter=
```

Step 7 screenshots, 2

If agent process emits an error about an assertion in Mmap, this means that memory needs to be defragmented, see the command below

```
(vrouter-agent)[root@opensdn26-VirtualBox /]$ /usr/bin/usr/bin/contrail-vrouter-agent
Config file </etc/contrail/contrail-vrouter-agent.conf> parsing completed.
INFO: wait vhost0 to be initilaized... 0/60
INFO: vhost0 is ready.
contrail-vrouter-agent: controller/src/vnsw/agent/vrouter/ksync/ksync_memory.cc:114: void KSyncMemory::Mmap(bool, void*, bool): Assertion `0' failed.
Aborted (core dumped)
(vrouter-agent)[root@opensdn26-VirtualBox /]$ cat /proc/buddyinfo
Node 0, zone      DMA      0      0      0      0      0      0      0      0      0      1      3
Node 0, zone    DMA32   2130    215    604    515    382    192     57      7      4      2      7
Node 0, zone   Normal   3140    998    336    197    110     39     17      6      8      0      0
(vrouter-agent)[root@opensdn26-VirtualBox /]$ exit
exit
opensdn26@opensdn26-VirtualBox:~$ sync && sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'
```

```
opensdn26@opensdn26-VirtualBox:~$ cat /proc/buddyinfo
Node 0, zone      DMA      0      0      0      0      0      0      0      0      0      1      3
Node 0, zone    DMA32  58773  35393  18935   9745   4159   1776    698    329    115     36     58
Node 0, zone   Normal  16370  13960   7224   4077   1866    525    134     35     16      4     11
opensdn26@opensdn26-VirtualBox:~$ sudo docker exec -ti agent bash
(vrouter-agent)[root@opensdn26-VirtualBox /]$ /usr/bin/contrail-vrouter-agent
Config file </etc/contrail/contrail-vrouter-agent.conf> parsing completed.
INFO: wait vhost0 to be initilaized... 0/60
INFO: vhost0 is ready.
dev_idx = 17, dev_name = vhost0
```

Answering from the controller to the agent

- This slide corresponds to step 8 from steps.md.
- When an agent connects to a controller, it sends greetings to it (first line) and waits for the reply.
- Send the controller’s reply (controller-reply.xml) using nc:
  - copy controller-reply.xml into the exchange buffer (Ctrl+C);
  - paste it into the terminal with “nc” and press enter;
- The agent will reply with <pubsub …> </pubsub>.

```
opensdn26@opensdn26-VirtualBox:~$ sudo nc -4kl -s 10.0.2.15 -p 5269
<stream:stream from="opensdn26-VirtualBox" to="network-control@contrailsystems.com" version="1.0" xml:lang="en" xmlns="" xmlns:stream="http://etherx.jabber.org/streams" >

<stream:stream from="network-control@contrailsystems.com" to="opensdn26-VirtualBox" id="++123" version="1.0" xml:lang="en" xmlns="" xmlns:stream="http://etherx.jabber.org/streams" >
Â<iq type="set" from="opensdn26-VirtualBox" to="network-control@contrailsystems.com/config">
<pubsub xmlns="http://jabber.org/protocol/pubsub">
<subscribe node="virtual-router:default-global-system-config:opensdn26-VirtualBox" />
</pubsub>
</iq>
```

Sending IF-MAP update to the agent

- This slide corresponds to step 9 from steps.md.
- If the agent answers at the previous step, send updates prepared at the step 2 (XML files) **in the next order**: global-config.xml, default-project.xml, admin-project.xml, cont1.xml, cont2.xml.
- Each XML **must be submitted** to “nc” **without large time delays** (several seconds) to avoid disconnection between the controller (nc) and the agent.
- In case of disconnection, the previous and current steps (7,8 and 9) must be repeated.

```
<iq type="set" from="opensdn26-VirtualBox" to="network-control@contrailsystems.com/bgp-peer" id="collection_120" >
<pubsub xmlns="http://jabber.org/protocol/pubsub">
<collection node="default-domain:default-project:ip-fabric:ip-fabric">
<associate node="25/242/08:00:27:13:0a:e3,10.0.2.15/32" />
</collection>
</pubsub>
</iq>

<message from="opensdn26-VirtualBox" to="network-control@contrailsystems.com/bgp-peer">
        <event xmlns="http://jabber.org/protocol/pubsub">
<items node="0/0/EndOfRib"></items>
        </event>
```

Registering virtual ports

- This slide corresponds to step 10 from steps.md.
- To register ports from the host OS in OpenSDN vRouter Agent, next commands must be issued:
  - tut-agent/scripts/run-port-control-cont1
  - tut-agent/scripts/run-port-control-cont2

```
opensdn26@opensdn26-VirtualBox:~$ tut-agent/scripts/run-port-control-cont2
/home/opensdn26/./vrouter-port-control:158: SyntaxWarning: invalid escape sequence '\s'
  regex = re.compile("\s+(?=\-\-[_a-zA-Z0-9]+=\".*?\")")
/home/opensdn26/./vrouter-port-control:161: SyntaxWarning: invalid escape sequence '\s'
  regex = re.compile("\s+(?=\-\-)")
JSON data written to /var/lib/contrail/ports/00000000-0000-0000-0000-000000000003
Request : {"id": "00000000-0000-0000-0000-000000000003", "instance-id": "00000000-0000-0000-0000-000000000016", "ip-address": "10.10.10.22", "ip6-address":
, "vn-id": "00000000-0000-0000-0000-000000000067", "display-name": "cont2", "vm-project-id": "", "mac-address": "86:64:86:cc:8b:d3", "system-name": "veth2"
type": 0, "rx-vlan-id": -1, "tx-vlan-id": -1, "vhostuser-mode": 0, "author": "/home/opensdn26/./vrouter-port-control", "time": "2026-03-09 22:16:07.342602"
opensdn26@opensdn26-VirtualBox:~$
```

Checking RIB/FIB

- This slide corresponds to step 11 from steps.md.
- To check an agent’s state, run any browser on the host with the agent and execute next address:
  - http://127.0.0.1:8085

Manual RIB checks

- RIB is displayed at the address:
  - http://127.0.0.1:8085/Snh_VrfListReq?name=
  - The result must include default-domain:admin:vn-A:vn-A.
- List of interfaces can be checked with
  - http://127.0.0.1:8085/Snh_ItfReq
  - The result must include veth1 and veth2 lines.

Manual FIB checks

- FIB can be checked using opensdn-tools container (see vRouter Forwarder basic tutorial for the reference, https://github.com/mkraposhin/opensdn-forwarder-basic-tutorial): utilities rt, nh, vif.
- Examples of correct output are given on the next slides.

rt output

```
(opensdn-tools)[root@opensdn26-VirtualBox /]$ rt --dump 2 --family bridge
Flags: L=Label Valid, Df=DHCP flood, Mm=Mac Moved, L2c=L2 Evpn Control Word, N=New Entry, Ec=EvpnControlProcessing
vRouter bridge table 0/2
Index       DestMac                  Flags           Label/VNID      Nexthop           Stats
31264       0:0:5e:0:1:0                Df                    -            3               0
47304       8:0:27:13:a:e3              Df                    -            3               0
52176       86:64:86:cc:8b:d3                                 -           30              16
75792       1e:a4:32:81:d5:70                                 -           23              15
112924      ff:ff:ff:ff:ff:ff          LDf                    3           33               1
115240      2:0:0:0:0:2                 Df                    -           12               0
205564      2:0:0:0:0:1                 Df                    -           12               0
(opensdn-tools)[root@opensdn26-VirtualBox /]$
```

vif output

```
vif0/1      OS: vhost0 NH: 5
            Type:Host HWaddr:08:00:27:13:0a:e3 IPaddr:10.0.2.15
            Vrf:0 Mcast Vrf:65535 Flags:L3DEr QOS:-1 Ref:8
            RX packets:340  bytes:21313 errors:0
            TX packets:632  bytes:37158 errors:0
            Drops:32

vif0/2      OS: pkt0
            Type:Agent HWaddr:00:00:5e:00:01:00 IPaddr:0.0.0.0
            Vrf:65535 Mcast Vrf:65535 Flags:L3Er QOS:-1 Ref:3
            RX packets:254  bytes:22641 errors:0
            TX packets:324  bytes:34992 errors:0
            Drops:17

vif0/3      OS: veth1 NH: 20
            Type:Virtual HWaddr:00:00:5e:00:01:00 IPaddr:10.10.10.11
            Vrf:2 Mcast Vrf:2 Flags:L3L2DEr QOS:-1 Ref:6
            RX packets:16  bytes:1186 errors:0
            TX packets:16  bytes:1186 errors:0
            Drops:0

vif0/4      OS: veth2 NH: 27
            Type:Virtual HWaddr:00:00:5e:00:01:00 IPaddr:10.10.10.22
            Vrf:2 Mcast Vrf:2 Flags:L3L2DEr QOS:-1 Ref:6
            RX packets:16  bytes:1186 errors:0
            TX packets:16  bytes:1186 errors:0
            Drops:2
```

nh output (Vrf:2 corresponds to vn-A VRF table)

```
opensdn26@opensdn...   opensdn26@opensdn...   @opensdn26-Virtual...   root@4d591cf0973e

Id:18         Type:Vrf_Translate  Fmly:AF_BRIDGE  Rid:0  Ref_cnt:3          Vrf:2
              Flags:Valid, Etree Root,
              Vrf:2

Id:19         Type:Composite      Fmly:AF_BRIDGE  Rid:0  Ref_cnt:1          Vrf:2
              Flags:Valid, Multicast, Etree Root,
              Sub NH(label):

Id:20         Type:Encap          Fmly: AF_INET  Rid:0  Ref_cnt:4          Vrf:2
              Flags:Valid, Policy, Etree Root,
              EncapFmly:0806 Oif:3 Len:14
              Encap Data: 1e a4 32 81 d5 70 00 00 5e 00 01 00 08 00

Id:21         Type:Encap          Fmly: AF_INET  Rid:0  Ref_cnt:2          Vrf:2
              Flags:Valid, Etree Root,
              EncapFmly:0806 Oif:3 Len:14
              Encap Data: 1e a4 32 81 d5 70 00 00 5e 00 01 00 08 00

Id:22         Type:Encap          Fmly: AF_INET  Rid:0  Ref_cnt:2          Vrf:2
              Flags:Valid, Multicast, Etree Root,
              EncapFmly:0806 Oif:3 Len:14
              Encap Data: 1e a4 32 81 d5 70 00 00 5e 00 01 00 08 00

Id:23         Type:Encap          Fmly:AF_BRIDGE  Rid:0  Ref_cnt:4          Vrf:2
              Flags:Valid, Etree Root,
              EncapFmly:0806 Oif:3 Len:14
              Encap Data: 1e a4 32 81 d5 70 00 00 5e 00 01 00 08 00

Id:24         Type:Encap          Fmly:AF_BRIDGE  Rid:0  Ref_cnt:2          Vrf:2
              Flags:Valid, Policy, Etree Root,
              EncapFmly:0806 Oif:3 Len:14
```

```
opensdn26@opensdn...   opensdn26@opensdn...   @opensdn26-Virtual...   root@4d591cf0973e:/

              EncapFmly:0806 Oif:4 Len:14
              Encap Data: 86 64 86 cc 8b d3 00 00 5e 00 01 00 08 00

Id:28         Type:Encap          Fmly: AF_INET  Rid:0  Ref_cnt:2          Vrf:2
              Flags:Valid, Etree Root,
              EncapFmly:0806 Oif:4 Len:14
              Encap Data: 86 64 86 cc 8b d3 00 00 5e 00 01 00 08 00

Id:29         Type:Encap          Fmly: AF_INET  Rid:0  Ref_cnt:2          Vrf:2
              Flags:Valid, Multicast, Etree Root,
              EncapFmly:0806 Oif:4 Len:14
              Encap Data: 86 64 86 cc 8b d3 00 00 5e 00 01 00 08 00

Id:30         Type:Encap          Fmly:AF_BRIDGE  Rid:0  Ref_cnt:4          Vrf:2
              Flags:Valid, Etree Root,
              EncapFmly:0806 Oif:4 Len:14
              Encap Data: 86 64 86 cc 8b d3 00 00 5e 00 01 00 08 00

Id:31         Type:Encap          Fmly:AF_BRIDGE  Rid:0  Ref_cnt:2          Vrf:2
              Flags:Valid, Policy, Etree Root,
              EncapFmly:0806 Oif:4 Len:14
              Encap Data: 86 64 86 cc 8b d3 00 00 5e 00 01 00 08 00

Id:32         Type:Composite      Fmly:AF_BRIDGE  Rid:0  Ref_cnt:2          Vrf:2
              Flags:Valid, Encap, Etree Root,
              Sub NH(label): 23(0) 30(0)

Id:33         Type:Composite      Fmly:AF_BRIDGE  Rid:0  Ref_cnt:3          Vrf:2
              Flags:Valid, Multicast, Etree Root,
              Sub NH(label): 32(0)

(opensdn-tools)[root@opensdn26-VirtualBox /]$
```

Connectivity tests

- This slide corresponds to step 12 from steps.md.
- This step includes:
  - an ICMP test using “ping” command;
  - a UDP test using “nc” command.

ICMP test

- Run in the container 2:
  - ping 10.1.1.11

```
root@481a938c0a31:/# ping 10.1.1.11
PING 10.1.1.11 (10.1.1.11) 56(84) bytes of data.
64 bytes from 10.1.1.11: icmp_seq=1 ttl=64 time=0.067 ms
64 bytes from 10.1.1.11: icmp_seq=2 ttl=64 time=0.182 ms
64 bytes from 10.1.1.11: icmp_seq=3 ttl=64 time=0.174 ms
64 bytes from 10.1.1.11: icmp_seq=4 ttl=64 time=0.085 ms
64 bytes from 10.1.1.11: icmp_seq=5 ttl=64 time=0.085 ms
^C
--- 10.1.1.11 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4147ms
rtt min/avg/max/mdev = 0.067/0.118/0.182/0.049 ms
```

UDP test

- Run in the container 1:
  - nc -4ul -s 10.1.1.11 -p 12000
- Run in the container 2:
  - nc -4u 10.1.1.11 12000
- Send messages between containers. Reference results are on the next slide.

UDP test results

```
15: veth2c@if16: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 86:64:86:cc:8b:d3 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.1.1.22/24 scope global veth2c
       valid_lft forever preferred_lft forever
    inet6 fe80::8464:86ff:fecc:8bd3/64 scope link
       valid_lft forever preferred_lft forever
root@481a938c0a31:/# ping 10.1.1.11
PING 10.1.1.11 (10.1.1.11) 56(84) bytes of data.
64 bytes from 10.1.1.11: icmp_seq=1 ttl=64 time=0.085 ms
64 bytes from 10.1.1.11: icmp_seq=2 ttl=64 time=0.091 ms
^C
--- 10.1.1.11 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1049ms
rtt min/avg/max/mdev = 0.085/0.088/0.091/0.003 ms
root@481a938c0a31:/# nc -4u 10.1.1.11 12000
qqqq
vvvv
```

cont2

```
13: veth1c@if14: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 1e:a4:32:81:d5:70 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.1.1.11/24 scope global veth1c
       valid_lft forever preferred_lft forever
    inet6 fe80::1ca4:32ff:fe81:d570/64 scope link
       valid_lft forever preferred_lft forever
root@4d591cf0973e:/# nc -4ul -p 12000

^C
root@4d591cf0973e:/# nc -4ul -s 10.1.1.11 -p 12000
qqqq
vvvv
```

cont1